Reproducing and Analysing the TypiClust Active Learning Algorithm on CIFAR-10

Lupupa Chansa - https://github.com/lupnpa/MLF-Coursework-2

26 March 2026

1 Introduction to the Problem

The paper addresses a problem that arises with uncertainty estimates in active learning. In many active learning methods, samples are chosen to maximise the information gained by selecting points with the highest uncertainty. However, when very few samples are labelled, uncertainty is not a reliable measure as the model does not yet have a sufficient representation of the data. As a result, uncertainty estimates can be misleading and lead to poor sample selection. The TypiClust algorithm instead allows the model to learn the data distribution first, reducing the risk of being misled by unreliable estimates.

In some cases, uncertainty-based methods may perform worse than random sampling. This problem becomes significant in scenarios where labelled samples are scarce, such as low-budget learning regimes or domains where labelled data is difficult to obtain. Addressing this limitation is important in the broader field of machine learning, as many real-world applications involve large amounts of unlabelled data but limited labelled data, making efficient sample selection crucial for improving model performance.

2 Methodology

The algorithm first takes the input data and learns a representative transformation of the vector space such that similar inputs are mapped closer together, and dissimilar inputs are pushed further apart. Next, a clustering algorithm is applied to the data in this representation space. The algorithm identifies the structural distribution of the data and groups similar samples together.

For each cluster, the point that is most "typical" is selected. It's goal is to select a point that represents the cluster as well as possible. This selected sample is then queried for its label and added to the labelled dataset, where it can be used to train the model. Over multiple iterations, the model gradually improves as more representative samples are labelled.

Clustering is a key component of the method because it ensures diversity by selecting samples from different regions of the data distribution. The choice of clustering algorithm can influence the quality of the clusters, as different algorithms may better capture different data structures. Another key concept is the typicality measure, which identifies points located in dense regions of the data distribution. These points are considered representative of many other nearby samples and therefore provide useful information when labelled.

However, in very dense or symmetric clusters, highly typical samples may still be very similar to each other. This can reduce diversity among selected samples and lead to an inefficient use of the labelling budget.

3 Results

TypiClust consistently improves on random sampling, and both variants outperform all baseline strategies for active learning. The paper evaluates the algorithm under both fully supervised and semi-supervised training frameworks. TypiClust demonstrates strong performance in both settings, particularly with the low-label regime.

Method	CIFAR-10	CIFAR-100	ImageNet subset
Random	∼22%	∼10%	∼9%
CoreSet	∼20%	∼9%	∼6%
BADGE	∼22%	∼9%	∼8%
TypiClust-DC	∼29%	∼13%	–
TypiClust-RP	∼27%	∼14%	∼12%

Table 1: Accuracy comparison after six iterations where label budget $B = C$. Values approximated from Fig. 4 in the original paper.

4 Plots

In my reproduced experiments, TypiClust methods outperform uncertainty-based methods such as entropy and margin sampling, especially in earlier iterations. This aligns with the paper's claim that uncertainty estimates are unreliable when very few labels are available.

In addition, the introduction of a diversity penalty achieves slightly improved performance in several iterations (e.g., reaching 28.56%). These findings align with the paper's observation that relying solely on typicality may introduce redundancy.

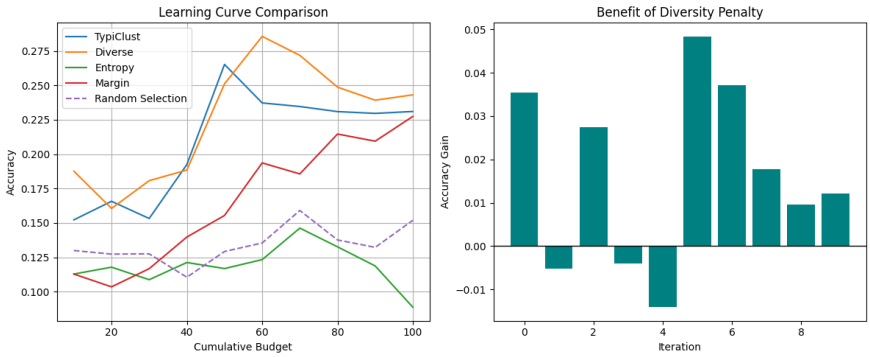


Figure 1: Accuracy comparison recreated from the original paper (left) Accuracy gained by diversity-aware modification of TypiClust (right)

5 Statistical Analysis

At the initial iteration, TypiClust achieved 15.22% accuracy, an approximate 35% improvement over both entropy and margin selection methods. At its peak it reaches an accuracy of 26.52%, compared to 15.19% for random sampling, a nearly 75% increased efficiency. These results support the original paper's claim that selecting representative samples significantly improves learning efficiency.

6 Conclusion

The paper addresses the cold-start problem in active learning, where the label budget is low, we see uncertainty-based methods are unreliable. It proposes TypiClust, which selects samples based on data structure rather than model uncertainty.

TypiClust improves learning efficiency when labelled data is scarce. This has important implications for real-world applications where data annotation is expensive or limited. The results also demonstrate that incorporating diversity into the selection process can further improve performance, particularly in later iterations. This suggests that balancing representativeness and diversity is important for effective active learning. Future work could explore improved strategies for diverse sample selection or stronger representation learning methods.

```
from google.colab import drive
drive.mount('/content/drive')
```

```
Mounted at /content/drive
```

```
import torch
import torchvision
import torchvision.models as models
import torchvision.transforms as transforms
import torch.optim as optim
import torch.nn as nn
import torch.nn.functional as F

import numpy as np
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.neighbors import NearestNeighbors
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
```

Model training pipeline for Resnet-18

I used SimCLR, an unsupervised learning framework that learns image representations without labels. The model trains to maximise similarity between two differently augmented views of the same image while separating them from other samples.

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

clr_transform = transforms.Compose([
    transforms.RandomResizedCrop(224),
    transforms.RandomHorizontalFlip(),
    transforms.ColorJitter(0.4, 0.4, 0.4, 0.1),
    transforms.RandomGrayscale(p=0.2),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

transform = transforms.Compose([
    transforms.Resize(224),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5]*3, std=[0.5]*3)
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transforms.ToTensor(
testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
```

```
100%|██████████| 170M/170M [00:11<00:00, 14.6MB/s]
```

```
class SimCLR(nn.Module):
    def __init__(self):
        super().__init__()
        base_model = models.resnet18(weights=None)
        self.encoder = nn.Sequential(*list(base_model.children())[:-1])
        self.projector = nn.Sequential(
            nn.Linear(512, 256),
            nn.ReLU(),
            nn.Linear(256, 128)
        )

    def forward(self, x):
        h = self.encoder(x)
        h = h.view(h.size(0), -1)
        z = self.projector(h)
        return h, z

def nt_xent_loss(z_i, z_j, temperature=0.5):

    z = torch.cat([z_i, z_j], dim=0)
    z = F.normalize(z, dim=1)

    sim = torch.matmul(z, z.T) / temperature

    batch_size = z_i.size(0)
    labels = torch.arange(batch_size).to(z.device)
    labels = torch.cat([labels + batch_size, labels])

    mask = torch.eye(2 * batch_size, dtype=torch.bool).to(z.device)
    sim = sim.masked_fill(mask, -9e15)
```

```
    return F.cross_entropy(sim, labels)

model_path = '/content/drive/MyDrive/resnet_model_simclr.pth'
model = SimCLR().to(device)
```

```
import os

if os.path.exists(model_path):
    print("Loading model...")
    model.load_state_dict(torch.load(model_path, map_location=device))
else:

    optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
    trainloader = torch.utils.data.DataLoader(trainset, batch_size=64, shuffle=True)

    for epoch in range(50):
        epoch_loss = 0

        for images, _ in trainloader:

            x_i = torch.stack([
                clr_transform(transforms.ToPILImage()(img.cpu()))
                for img in images
            ]).to(device)

            x_j = torch.stack([
                clr_transform(transforms.ToPILImage()(img.cpu()))
                for img in images
            ]).to(device)

            _, z_i = model(x_i)
            _, z_j = model(x_j)

            loss = nt_xent_loss(z_i, z_j)

            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            epoch_loss += loss.item()

        print(f"Epoch {epoch+1}, Loss: {epoch_loss:.4f}")

        torch.save(model.state_dict(), model_path)
```

Show hidden output

```
def extract_embeddings(dataset):
    model.eval()

    embeddings, labels = [], []
    loader = torch.utils.data.DataLoader(dataset, batch_size=64, shuffle=False)

    with torch.no_grad():
        for img, label in loader:
            img = img.to(device)
            h, _ = model(img)
            h = h.view(h.size(0), -1)

            embeddings.append(h.cpu().numpy())
            labels.append(label.numpy())

    return np.concatenate(embeddings), np.concatenate(labels)

subset_size = 10000
train_subset = torch.utils.data.Subset(trainset, range(subset_size))
Z, y = extract_embeddings(train_subset)

Z_test, y_test = extract_embeddings(testset)

print("Embeddings:", Z.shape)
```

Show hidden output

Baselines for Accuracy Comparison

```
def random_sampling(unlabelled, k=10):
    return np.random.choice(len(unlabelled), k, replace=False)
```

```
def entropy_sampling(model, Z, k):
    probs = model.predict_proba(Z)
    entropy = -np.sum(probs * np.log(probs + 1e-10), axis=1)
    return np.argsort(entropy)[-k:]

def margin_sampling(model, Z, k):
    probs = model.predict_proba(Z)
    sorted_probs = np.sort(probs, axis=1)
    margin = sorted_probs[:, -1] - sorted_probs[:, -2]
    return np.argsort(margin)[:k]
```

Algorithm Modification and Evaluation

I thought of implementing a diversity-aware TypiClust to address the issue of redundant sample selection. While the original method selects highly typical samples from dense regions, these samples may limit the information gained, which is a concern mentioned in the paper (3.2. Proposed Strategy: Typical Clustering (b)). To mitigate this, introducing a diversity penalty discourages selecting samples close to previously chosen points.

```
def compute_typicality(cluster_points, k=10):
    if len(cluster_points) <= k:
        k = max(1, len(cluster_points) - 1)

    nbrs = NearestNeighbors(n_neighbors=k).fit(cluster_points)
    distances, _ = nbrs.kneighbors(cluster_points)
    return 1 / (distances.mean(axis=1) + 1e-6)
```

```
def typiclust_selection(data, n_clusters):
    kmeans = KMeans(n_clusters=n_clusters, n_init=10).fit(data)
    labels = kmeans.labels_
    selected = []

    for cluster_id in range(n_clusters):
        indices = np.where(labels == cluster_id)[0]
        if len(indices) == 0: continue

        typicality = compute_typicality(data[indices])
        best = indices[np.argmax(typicality)]
        selected.append(best)

    return selected
```

```
def diversity_penalty(point, selected_points):
    if len(selected_points) == 0: return 0
    distances = np.linalg.norm(selected_points - point, axis=1)
    return 1 / (distances.mean() + 1e-6)

def typiclust_diverse(data, n_clusters, lam=0.5):
    kmeans = KMeans(n_clusters=n_clusters, n_init=10).fit(data)
    labels = kmeans.labels_
    selected_indices, selected_points = [], []

    for cluster_id in range(n_clusters):
        indices = np.where(labels == cluster_id)[0]
        if len(indices) == 0: continue

        cluster_points = data[indices]
        typicality = compute_typicality(cluster_points)

        scores = []
        for i, point in enumerate(cluster_points):
            penalty = diversity_penalty(point, np.array(selected_points))
            scores.append(typicality[i] - lam * penalty)

        best_idx = indices[np.argmax(scores)]
        selected_indices.append(best_idx)
        selected_points.append(data[best_idx])

    return selected_indices
```

```
labelled_rand = []
unlabelled_rand = list(range(len(Z)))
accuracies_rand = []

for iteration in range(10):

    chosen = np.random.choice(len(unlabelled_rand), 10, replace=False)
    chosen_global = [unlabelled_rand[i] for i in chosen]
```

```
    labelled_rand.extend(chosen_global)
    unlabelled_rand = [i for i in unlabelled_rand if i not in chosen_global]

    clf = LogisticRegression(max_iter=1000).fit(Z[labelled_rand], y[labelled_rand])

    acc = accuracy_score(y_test, clf.predict(Z_test))
    accuracies_rand.append(acc)

    print(f"Random Iteration {iteration}: {acc:.4f}")
```

Show hidden output

```
labelled_ent = []
unlabelled_ent = list(range(len(Z)))
accuracies_ent = []

for iteration in range(10):

    if len(labelled_ent) == 0:
        labelled_ent.extend(unlabelled_ent[:10])
        unlabelled_ent = unlabelled_ent[10:]
    else:
        clf = LogisticRegression(max_iter=1000).fit(Z[labelled_ent], y[labelled_ent])
        idx = entropy_sampling(clf, Z[unlabelled_ent], 10)
        chosen = [unlabelled_ent[i] for i in idx]

        labelled_ent.extend(chosen)
        unlabelled_ent = [i for i in unlabelled_ent if i not in chosen]

    clf = LogisticRegression(max_iter=1000).fit(Z[labelled_ent], y[labelled_ent])
    acc = accuracy_score(y_test, clf.predict(Z_test))
    accuracies_ent.append(acc)

    print(f"Entropy Iteration {iteration}: {acc:.4f}")
```

Show hidden output

```
labelled_mar = []
unlabelled_mar = list(range(len(Z)))
accuracies_mar = []

for iteration in range(10):

    if len(labelled_mar) == 0:
        labelled_mar.extend(unlabelled_mar[:10])
        unlabelled_mar = unlabelled_mar[10:]
    else:
        clf = LogisticRegression(max_iter=1000).fit(Z[labelled_mar], y[labelled_mar])
        idx = margin_sampling(clf, Z[unlabelled_mar], 10)
        chosen = [unlabelled_mar[i] for i in idx]

        labelled_mar.extend(chosen)
        unlabelled_mar = [i for i in unlabelled_mar if i not in chosen]

    clf = LogisticRegression(max_iter=1000).fit(Z[labelled_mar], y[labelled_mar])
    acc = accuracy_score(y_test, clf.predict(Z_test))
    accuracies_mar.append(acc)

    print(f"Margin Iteration {iteration}: {acc:.4f}")
```

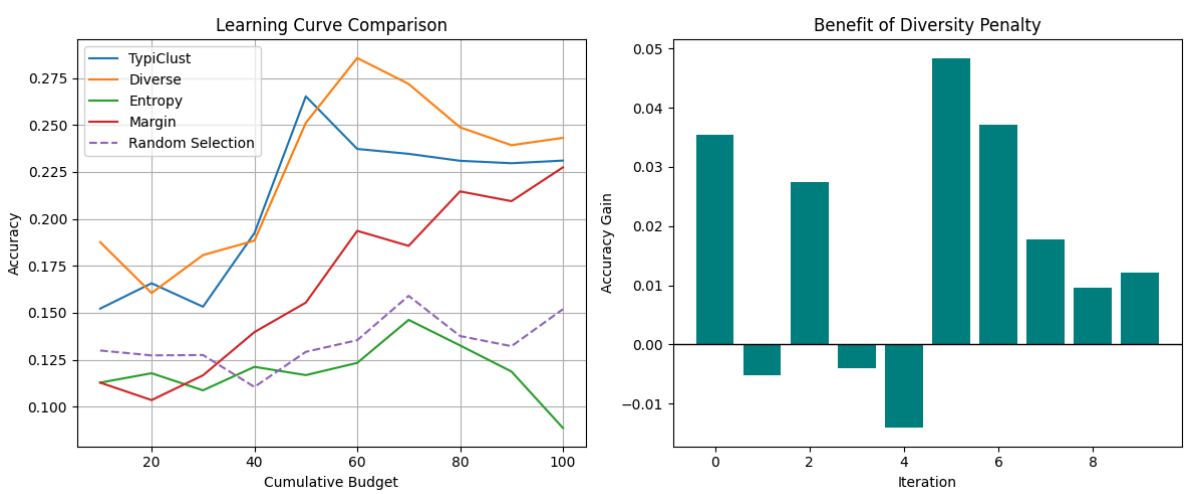
Show hidden output

```
labelled_typ = []
unlabelled_typ = list(range(len(Z)))
accuracies = []

for iteration in range(10):

    chosen = typiclust_selection(Z[unlabelled_typ], 10)
    chosen_global = [unlabelled_typ[i] for i in chosen]

    labelled_typ.extend(chosen_global)
    unlabelled_typ = [i for i in unlabelled_typ if i not in chosen_global]

    clf = LogisticRegression(max_iter=1000).fit(Z[labelled_typ], y[labelled_typ])
    acc = accuracy_score(y_test, clf.predict(Z_test))
    accuracies.append(acc)

    print(f"TypiClust Iteration {iteration}: {acc:.4f}")
```

Show hidden output

```
labelled_div = []
unlabelled_div = list(range(len(Z)))
accuracies_mod = []

for iteration in range(10):

    chosen = typiclust_diverse(Z[unlabelled_div], 10)
    chosen_global = [unlabelled_div[i] for i in chosen]

    labelled_div.extend(chosen_global)
    unlabelled_div = [i for i in unlabelled_div if i not in chosen_global]

    clf = LogisticRegression(max_iter=1000).fit(Z[labelled_div], y[labelled_div])
    acc = accuracy_score(y_test, clf.predict(Z_test))
    accuracies_mod.append(acc)

    print(f"Diverse Iteration {iteration}: {acc:.4f}")
```

Show hidden output

Comparison Plots for Final Results

```
plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)

x = [(i+1)*10 for i in range(len(accuracies))]

plt.plot(x,accuracies, label="TypiClust")
plt.plot(x,accuracies_mod, label="Diverse")
plt.plot(x,accuracies_ent, label="Entropy")
plt.plot(x,accuracies_mar, label="Margin")
plt.plot(x,accuracies_rand, label='Random Selection', linestyle='--')
plt.xlabel("Cumulative Budget")
plt.ylabel("Accuracy")
plt.title("Learning Curve Comparison")
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
diff = np.array(accuracies_mod) - np.array(accuracies)
plt.bar(range(len(diff)), diff, color='teal')
plt.axhline(0, color='black', lw=1)
plt.xlabel("Iteration")
plt.ylabel("Accuracy Gain")
plt.title("Benefit of Diversity Penalty")

plt.tight_layout()
plt.show()
```